Mocha

The Mocha library is the one Rails uses for its own testing, so it seems the natural choice for a test-double library to use with Minitest. Full Mocha docs are available online.[3]

Installing Mocha

Mocha needs to be installed after Minitest, which requires a slight indirection to ensure your Rails app does the right thing.

First, in the Gemfile's :test group, add the Mocha gem:

```
gem "mocha"
```

Then, inside the test/test_helper.rb file, you manually require Mocha at any point in the file after the rails/test_help library is loaded:

```
require "mocha/mini_test"
```

At that point you should be good to go. Please see Chapter 7, *Using Test Doubles as Mocks and Stubs*, on page 129, for a full discussion of test doubles. Here I'll cover only Mocha syntax.

In Mocha you can call stub to create a full double that is just a set of stubbed methods; stub is available throughout your test cases.

```
test "here's a sample stub" do
  stubby = stub(name: "Paul", weight: 100)
  assert_equal("Paul", stubby.name)
end
```

Mocha has verification syntax for full doubles. In Mocha you use responds_like and responds_like_instance_of to trigger verification that the object matches a given object's API. You might do so like this:

```
stubby = stub(size: 3)
stubby.responds_like(Task.new)
```

Or this:

```
stubby = stub(size: 3)
stubby.respond_like_instance_of(Task)
```

In either case, if you called stubby with a method that is not defined for the Task class—say, stubby.due_date—Mocha will raise a NoMethodError. Internally this uses the Ruby respond_to? method, so if you're using dynamically generated

3. http://gofreerange.com/mocha/docs